

LLMs & Ontology Engineering

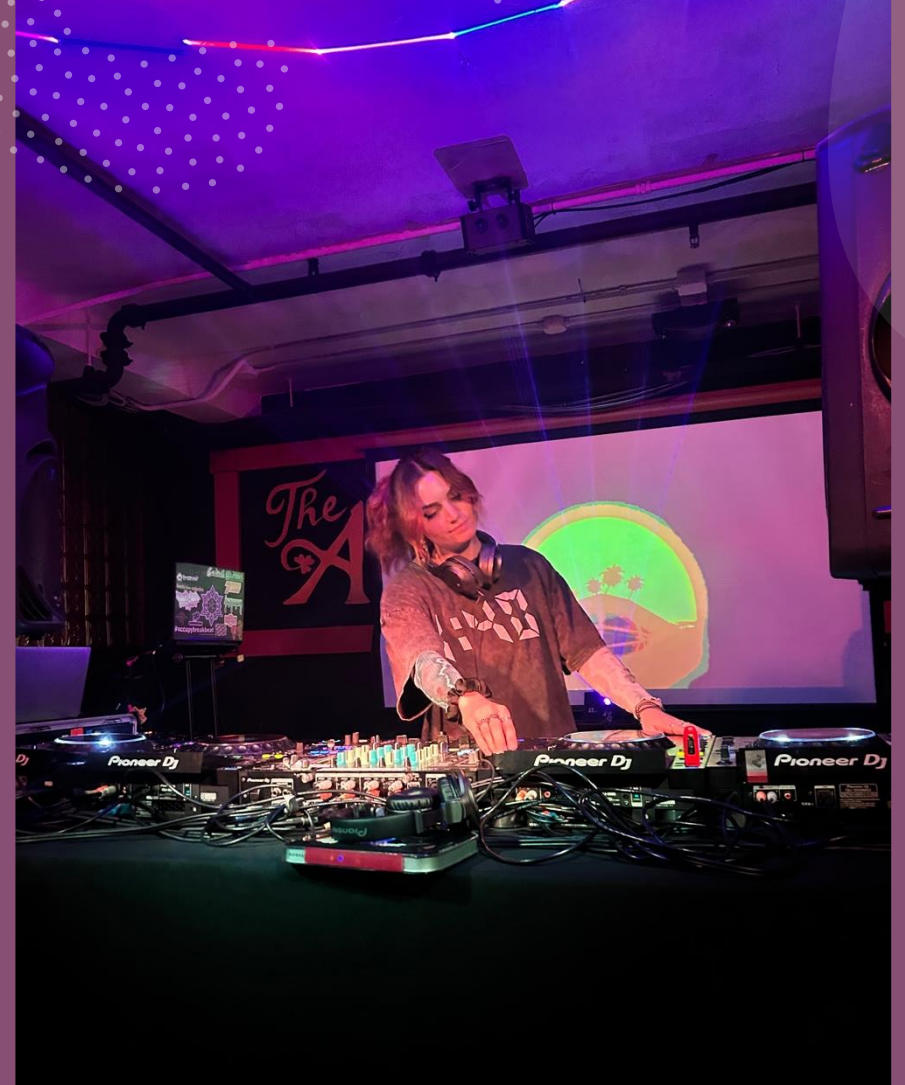
By Sydney Cohen (SydCo)

Hello, a bit about me

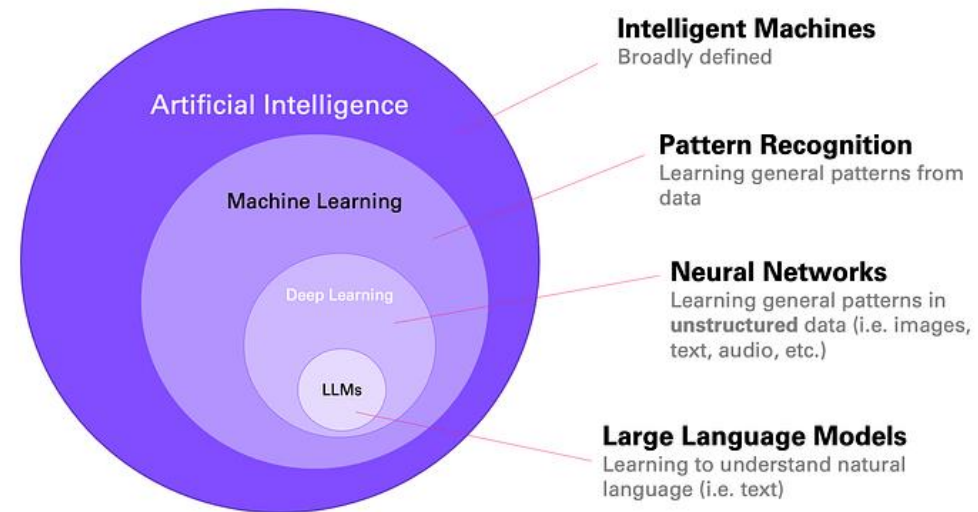
- NU class of '21
 - degree in Social Policy (lol)
- Life things and then John Beverley
- Experimenting with OBO Foundry and IDO extensions
- 6-month data science boot camp with NU/Trilogy
- Oh no, need monies, became a data scientist (UL Standards & Engagement)
- Tricked both myself & Johns Hopkins APL into believing I was an “Applied Ontologist”
- Went rogue, started my own SydCo Ontology Consulting LLC, working with NCOR and OntosCosmos now

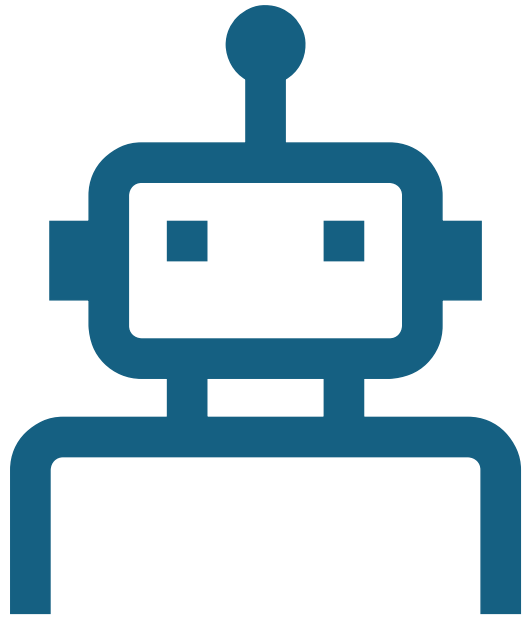
Some fun facts

- Lover of sci-fi and weird stories
- Proud owner of 2 cats
- I love using inconsistent slide styles
- Music enthusiast, occasional DJ



What are
LLMs (I'm
sorry but I
have to)





Okay, That's Cool SydCo, but I hate AI Slop

- Me too! We can be friends
- Skepticism SHOULD be the angle you take when using automated tools on a technology that is defined in many ways by its ability to *curate* human wisdom into human and machine-readable artifacts
 - The episistemic implications are non-trivial
- Anything you build with an LLM is only going to be as good as your ability to understand and validate it

Things I've Used LLMs For

Initial domain exploration (you want me to make an ontology about *aircraft*? Ope)

Coding frontends for tooling I've built (Excel spreadsheet outputs are not always the answer)

Coding backends faster (like, turning python scripts into FastAPI endpoints)

Coding one-off helper scripts for specific, repetitive ontology tasks

Exploring new codebases for tooling I'm considering using

As a step (tool call) in a broader pipeline such as RAG

High level notes about using coding agents

Existing code influences future code

It leaves #comments

Special (interesting) error handling

This guy > 

Unanticipated code choices

Function spamming

Documentation uses

Looooong files

More notes for using LLMs for generating ontology content

- Do as much of the pipeline as possible using anything BUT LLMs. Leverage algorithms that you have more control over first
- When generating code from LLMs, read over the code, line by line
 - Don't assume it's doing the thing you told it to do
- If your idea was a bad idea in the first place, the LLM isn't gonna tell you
- Token optimization matters beyond cost. LLMs 'spend' attention on things



In music,
repetition
legitimizes...

In a pipeline that uses LLMs, each stage should **reduce uncertainty** before passing work to the next stage.

Deterministic algorithms reduce structural uncertainty by filtering, validating, and enriching the graph.

Machine learning reduces semantic uncertainty by identifying the most relevant concepts or subgraphs.

LLMs synthesize, explain, generate, or ‘reason’ over the curated information.

Determine what is known, identify what might be relevant, then use LLMs to help interpret and communicate what it means.

Final thoughts

- LLMs are a useful tool in the belt of an ontology developer
 - It can compensate for a lack of coding skills, as it has many times in my experience (with caveats we've discussed)
 - It can speed up tedious technical development ("how do I connect to this relational database in Python?") to free up time for the thinky stuff ("What makes this mushroom different than the other one? How do I model the ability to fly?")
 - It can be an essential component of automated systems, when validated

Don't ask	an LLM to query what SPARQL can query.
Don't ask	an LLM to validate what SHACL can validate.
Don't ask	an LLM to infer what an OWL reasoner can infer.
Don't ask	an LLM to retrieve what embeddings can retrieve.
Ask	the LLM to do what only it can do: integrate, explain, and generate.